A Guide to Computational Mechanics

John S Azariah
University of Technology, Sydney
john.azariah@student.uts.edu.au

January 23, 2026

Abstract

How do we find structure in randomness? This guide introduces *Computational Mechanics*, a framework for inferring causal models from time-series data. We build up from basic Information Theory to the construction of ϵ-machines—optimal predictors of the future. We demonstrate these concepts using the "Golden Mean" process and the Logistic Map, verifying theoretical bounds along the way. This document is generated from a reproducible pipeline using the `emic` Python library.

Contents

1 Introduction

Imagine you are watching a stream of data go by. It might be the ups and downs of a stock market ticker, the sequence of DNA bases in a genome, or the turbulent velocity of a fluid.

At first glance, it looks random. But is it?

We are often tasked with finding *patterns* in data. But what exactly is a pattern? In statistics, we talk about "correlation". In dynamical systems, we talk about "attractors". Computational Mechanics provides a different answer: a pattern is a structure that allows us to **predict**.

To understand a process is to be able to predict its future given its past. If a process is perfectly random (like a coin flip), the past tells us nothing about the future. If it is periodic (like a clock), the past tells us everything. Most interesting processes lie somewhere in between: they are *structured stochastic processes*.

Intrinsic Computation

Computational Mechanics views nature not just as a system of energy, but as a system of *information processing*. The universe computes. A process "stores" information from the past (memory) and "processes" it to generate the future. Our goal is to reverse-engineer this computation from the data alone.

This guide will walk you through the framework of Computational Mechanics [2]. We will not just learn definitions; we will construct the optimal predictive model—the ϵ-machine—from scratch. We will see how to measure the "memory" of a process and verify that our models are as simple as possible, but no simpler.

2 Preliminaries: The Language of Uncertainty

Before we can talk about complexity and structure, we need a language to talk about uncertainty. That

language is Information Theory.

2.1 Entropy: A Measure of Surprise

The foundational quantity is Shannon Entropy, denoted H. It measures the average uncertainty or "surprise" inherent in a random variable.

If we have a random variable X that can take values x from an alphabet $\mathcal{A}$ with probability $P(x)$, the entropy is:

$$H[X] = -\sum_{x \in \mathcal{A}} P(x) \log_2 P(x) \tag{1}$$

We measure entropy in **bits**.

Why Bits?

Shannon chose base 2 for his logarithms, giving us the *bit* (binary digit). A bit represents a choice between two equally likely alternatives. However, other bases are possible:

- Base e gives the *nat*, used in thermodynamics [3].
- Base 10 gives the *ban* or *hartley* [1].

We use bits because they map intuitively to digital logic.

Key Intuition

Think of entropy as the number of yes/no questions you need to ask on average to determine the value of X. If a coin is fair, you need 1 question ("Is it Heads?"). $H = 1$ bit. If the coin is rigged to always be Heads, you need 0 questions. $H = 0$ bits.

2.2 Joint and Conditional Entropy

What if we have two variables, X and Y?

- **Joint Entropy** $H[X, Y]$: Uncertainty about the pair (X, Y).
- **Conditional Entropy** $H[X|Y]$: Uncertainty about X, given that we already know Y.

$$H[X|Y] = H[X, Y] - H[Y] \tag{2}$$

"The uncertainty about X given Y is the total uncertainty minus the uncertainty we resolved by learning Y."

2.3 Mutual Information

Perhaps the most important quantity for us is **Mutual Information** (I). It measures how much information Y provides about X (and vice versa).

$$I[X; Y] = H[X] - H[X|Y] \tag{3}$$

This is the reduction in uncertainty about X gained by observing Y. If X and Y are independent, knowing Y tells us nothing about X, so $I = 0$.

3 The Prediction Game

Now let's apply information theory to time. We view a stochastic process as a chain of random variables over time:

$$\ldots, X_{-2}, X_{-1}, X_0, X_1, X_2, \ldots$$

At any moment $t = 0$, we stand between two worlds:

- The **Past** (History): $\overleftarrow{X} = \ldots X_{-2} X_{-1}$
- The **Future**: $\vec{X} = X_0 X_1 X_2 \ldots$

3.1 The Goal of Modeling

Our goal is simple: predict the Future given the Past. We want to know the conditional distribution $P(\vec{X}|\overleftarrow{X})$.

However, the Past is infinitely long. We cannot just write down a lookup table for every possible infinite history. We need to *compress* the past. We need to find a summary of the past that is just as good as the full past for predicting the future.

3.2 Causal States

Two different histories, $\overleftarrow{x}$ and $\overleftarrow{x}'$, might lead to the exact same predictions. For example, in a periodic process "ABABA...", seeing "AB" implies the next symbol is "A". Seeing "ABAB" also implies the next symbol is "A". The histories are different, but their predictive implications are identical.

We say two histories are **causally equivalent** ($\sim_\epsilon$) if they have the same conditional distribution over futures:

$$\overleftarrow{x} \sim_\epsilon \overleftarrow{x}' \iff P(\vec{X}|\overleftarrow{X} = \overleftarrow{x}) = P(\vec{X}|\overleftarrow{X} = \overleftarrow{x}') \tag{4}$$

Occam's Razor

This equivalence relation is the mathematical embodiment of Occam's Razor. It groups distinct histories into the *same* state if they don't make a difference to the future. This ensures we never create a state unless it is theoretically necessary.

Key Intuition

If knowing you had coffee for breakfast changes your probability of being productive today, then "Coffee" and "No Coffee" are *causally distinct* states. If the color of your socks does not change that probability, then "Red Socks" and "Blue Socks" are *causally equivalent*.

The sets of equivalent histories are called **Causal States**, denoted $\mathcal{S}$.

3.3 The Epsilon Machine

The ϵ**-machine** is the model constructed from these causal states. It consists of: 1. The set of Causal States $\mathcal{S}$. 2. The transition rules between them.

It is the unique, minimal, optimal predictor of the process.

4 A Concrete Example: The Golden Mean

Let's make this concrete with a specific process: the "Golden Mean". This process generates binary sequences (0s and 1s) with a specific rule:

"No two consecutive 0s are allowed."

If we see a 0, the next symbol *must* be a 1. If we see a 1, the next symbol can be 0 or 1 (with equal probability, say).

Why "Golden Mean"?

The name comes from the growth rate of the number of allowed sequences. The number of valid binary strings $N(L)$ of length L containing no "00" follows the Fibonacci sequence. For large L, $N(L)$ grows as ϕ^L, where $\phi = \frac{1+\sqrt{5}}{2} \approx 1.618$ is the Golden Ratio.

4.1 Inferring the Structure

We run the CSSR algorithm (Causal State Splitting Reconstruction) on a generated sequence of 100,000 symbols. The algorithm looks at the statistics of subsequences and groups histories that lead to similar future probabilities.

4.2 Reading the Machine

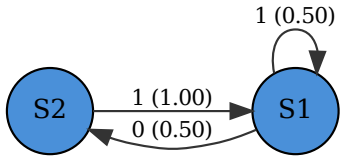
Look at the graph in Figure 1.

- **States**: There are two states. Let's call them A and B.

Code Listing 1: The Pipeline

```
from emic.sources import GoldenMeanSource
from emic.sources.transforms import TakeN
from emic.inference import CSSR, CSSRConfig
from emic.analysis import analyze

# 1. Define the source
source = GoldenMeanSource(p=0.5)

# 2. Configure the algorithm
cssr = CSSR(CSSRConfig(max_history=10))

# 3. The Inference Pipeline
# Source >> Transform >> Algorithm
result = source >> TakeN(100000) >> cssr

# 4. Analysis
summary = analyze(result.machine)
print(f"States: {summary.num_states}")
print(f"C_mu: {summary.statistical_complexity:.2f} bits")
```

Figure 1: The ϵ-machine for the Golden Mean Process.



- **Transitions**:
  - From A, we can see '1' (loop back to A) or '0' (go to B).
  - From B, we *only* see '1' (and go back to A).

Wait, does this match our rule?

- State A represents the condition "Last symbol was a 1". Since the last symbol was 1, we are free to emit 0 or 1 next.
- State B represents the condition "Last symbol was a 0". Since the last symbol was 0, the rule says we *must* emit a 1. So there is only one arrow leaving B.

The ϵ-machine has automatically discovered the "No 00" rule just from the data! It identified that "histories ending in 0" have different predictive consequences than "histories ending in 1".

5 Measures of Complexity

Once we have the ϵ-machine, we can calculate intrinsic properties of the process that go beyond simple statistics.

5.1 Definitions

We care about two fundamental quantities: structure and randomness.

Statistical Complexity (C_μ)
How much memory does the process need to store? It is defined as the Shannon entropy of the causal states themselves.

$$C_\mu = H[\mathcal{S}] = -\sum_{\sigma \in \mathcal{S}} \pi(\sigma) \log_2 \pi(\sigma) \quad (5)$$

where $\pi(\sigma)$ is the probability of being in state σ.

Entropy Rate (h_μ)
How random is the next symbol, given the past? It is the average uncertainty of the next symbol, averaged over the causal states.

$$h_\mu = H[X|\mathcal{S}] = \sum_{\sigma \in \mathcal{S}} \pi(\sigma) H[X|\sigma] \quad (6)$$

Intuition: Stored vs. Generated Information

These two measures give us a complete picture of the process's information budget:

1. C_μ is the information **stored** from the past. It represents *Structure*.
2. h_μ is the new information **generated** at each step. It represents *Randomness*.

Examples:

- **Clock**: High C_μ (many states to track), Zero h_μ (perfectly predictable).
- **Fair Coin**: Zero C_μ (no memory needed), High h_μ (maximum surprise).

5.2 Calculation

Let's compute these values for our Golden Mean process ($p = 0.5$).

Derivation for Golden Mean

1. Statistical Complexity (C_μ)
The stationary distribution over states is derived from the transition matrix. For the Golden Mean ($p = 0.5$):

$$\pi(A) = 2/3, \quad \pi(B) = 1/3$$

The entropy of this distribution is:

$$\begin{aligned} C_\mu &= -\sum \pi(\sigma) \log_2 \pi(\sigma) \\ &= -(2/3 \log_2 2/3 + 1/3 \log_2 1/3) \\ &\approx 0.918 \text{ bits} \end{aligned}$$

2. Entropy Rate (h_μ)
This is the average uncertainty of the next symbol given the current state.

- State A emits 0 or 1 with equal probability: $H[X|A] = 1$ bit.
- State B always emits 1: $H[X|B] = 0$ bits.

Weighted by state probability:

$$\begin{aligned} h_\mu &= \sum \pi(\sigma) H[X|\sigma] \\ &= (2/3 \times 1) + (1/3 \times 0) \\ &= 0.667 \text{ bits} \end{aligned}$$

5.3 Comparison of Processes

Different processes occupy different places on the complexity-entropy plane.

Table 1: Measures for Common Processes

Process	Description	C_μ	h_μ
Fair Coin	No memory, pure randomness.	0.00	1.00
Period-2	Alternating 0101... Needs 1 bit of memory.	1.00	0.00
Golden Mean	No '00'. A mix of structure and choice.	0.92	0.67

6 How Good is Our Prediction?

We have built machines that *seem* to capture the structure of a process. We calculated their complexity and entropy. But how do we know if they are actually *good* models?

Shalizi and Crutchfield (2001) give us two powerful guarantees—two "Fundamental Theorems" of Computational Mechanics.

6.1 Theorem 1: Prescience (Accuracy)

The first question is simple: **Does our model predict the future as well as the past allows?**

Imagine an Oracle who remembers the *entire* infinite history of the universe ($\overleftarrow{X}$). We, on the other hand, only know the current causal state ($\mathcal{S}$). Theorem 1 states that valid causal states are **prescient**:

$$P(\vec{X}|\mathcal{S}) = P(\vec{X}|\overleftarrow{X})$$

In English: *Knowing the causal state is just as good as knowing the entire history.* You lose zero predictive power by compressing the history into a state.

Key Intuition

The Cost of Being Wrong: KL Divergence
To measure how wrong a model is, we use the **Kullback-Leibler (KL) Divergence**. Think of it as a "distance" between two probability distributions.
If the true distribution of future symbols is P and our model predicts Q, the KL divergence $D_{KL}(P||Q)$ tells us how many extra bits we need to correct our mistake.

- If $D_{KL} = 0$, our model is perfect.
- If $D_{KL} > 0$, our model is missing something.

We can verify this theorem numerically. In `emic`, we can sweep through different complexity levels (by adjusting the significance level α of the statistical test) and measure the error.

Code Listing 2: Checking Optimality

```
# Does adding more states reduce error?
results = []
for alpha in [0.001, 0.01, 0.1, 0.5]:
    # Infer machine with decreasing strictness
    machine = cssr.infer(data, alpha)
    # Measure complexity (C_mu) and error (excess entropy)
    results.append((machine.c_mu, machine.kl_divergence))
```

In Figure 2, we plot the prediction error (KL Divergence) against the number of states.

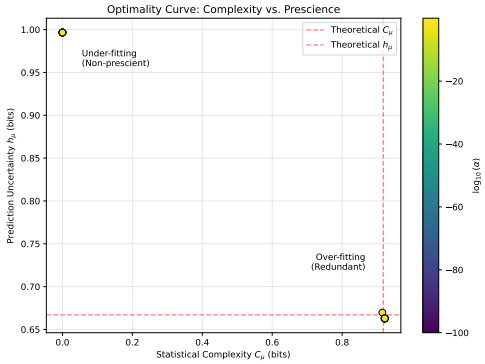
Notice the curve. As we add more states (moving to the right), the error drops. But once we hit the "correct" number of states (the ϵ-machine), the error hits zero. Adding more states after that point doesn't help—it just wastes memory.

6.2 Theorem 2: Minimality (Efficiency)

The second question is about efficiency: **Is this the smallest possible correct model?**

You could predict the future perfectly by simply memorizing every sequence of length 1000. That would be "prescient" (zero error), but it would be huge (2^{1000} states!).

Figure 2: The Optimality Curve



The x-axis is complexity (number of states), the y-axis is prediction error (D_{KL}). The ϵ-machine sits at the "elbow": minimal complexity for zero error.

Theorem 2 states that among all prescient models, the ϵ-machine is the **minimal** one. It has the smallest possible statistical complexity (C_μ).

$$C_\mu(\epsilon\text{-machine}) \leq C_\mu(\text{any other prescient model})$$

This confirms that the ϵ-machine is the **unique, simplest, optimal predictor** of a process.

7 From Rules to Chaos

We have seen simple processes like the Golden Mean, which require only a few bits of memory. But what happens when we look at complex physical systems?

One of the most surprising discoveries of the 20th century was **Chaos**: simple deterministic rules can produce behavior that looks completely random. Computational Mechanics gives us a toolkit to "measure" this chaos. It allows us to ask: *Is this randomness "simple" (like a coin flip) or "complex" (requiring memory)?*

7.1 The Logistic Map

Consider a simple model of population growth, the Logistic Map:

$$x_{n+1} = rx_n(1 - x_n)$$

where x_n is the population at year n, and r is the growth rate.

As we increase r, the behavior undergoes a dramatic transformation:

- **Order ($r < 3$)**: The population settles to a single stable value.

- **Periodicity ($r \approx 3.5$)**: The population bounces between 2, 4, 8... values.
- **Chaos ($r \approx 4.0$)**: The population jumps wildly and unpredictably.

7.2 The Structure of Chaos

We can convert this numerical data into symbols: '1' if the population is high ($x > 0.5$), '0' if low. Then, we use `emic` to infer the ϵ-machine at each value of r.

Code Listing 3: Scanning for Chaos

```
# Scan the parameter r
for r in np.linspace(3.5, 4.0, 100):
    data = logistic_map(r, size=10000)
    machine = cssr.infer(data)
    # How much memory does the process need?
    complexity = machine.statistical_complexity()
```

The results, shown in Figure 3, reveal the hidden architecture of chaos.

Key Intuition

The Edge of Chaos
Look at the blue curve (Statistical Complexity).

- In the **Ordered** regime, complexity is low. It's easy to predict a fixed point.
- In the **Chaotic** regime (far right), complexity is *also* low! If a system is completely random, you don't need memory to predict it—you just guess.
- The peaks occur at the **transition**, the "Edge of Chaos".

This tells us something profound: **Structure lives in the balance between order and randomness.**

8 Conclusion: The Physics of Information

We began this guide with a simple question: *How do we find pattern in randomness?*

Through the lens of Computational Mechanics, we have seen that "pattern" is not just a vague visual concept. It is a physical quantity that can be measured, derived, and understood.

1. **History matters:** We saw that the past influences the future, but not all pasts are created equal.
2. **States are summarized histories:** By grouping equivalent histories into **Causal States**, we found the minimal necessary information to predict the future.
3. **Machines can be inferred:** using algorithms like CSSR, we can reconstruct the ϵ-machine purely from data, as we did with the Golden Mean.
4. **Complexity is physically meaningful:** We discovered that structure (C_μ) peaks not in total order or total chaos, but at the phase transitions between them.

This framework transforms "Information" from a static quantity (bits in a hard drive) into a dynamical one (bits required to predict). Whether you are studying the string of DNA code, the fluctuations of a stock market, or the firing of neurons, the ϵ-machine gives you the intrinsic architecture of the process.

We invite you to use the `emic` library to explore your own data. The tools we used here—generating data, inferring states, calculating complexity—are available for you to discover the hidden structure in your own world.

Further Reading

- Shalizi, C. R., & Crutchfield, J. P. (2001). *Computational mechanics: Pattern and prediction, structure and simplicity*.
- Crutchfield, J. P. (2012). *Between order and chaos*. Nature Physics.

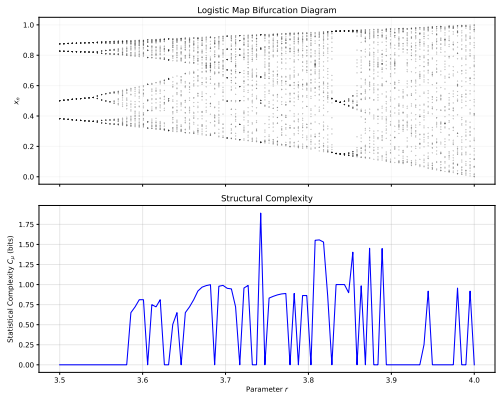
References

[1] Ralph VL Hartley. Transmission of information. *Bell System technical journal*, 7(3):535–563, 1928.

[2] Cosma Rohilla Shalizi and James P Crutchfield. Computational mechanics: Pattern and prediction, structure and simplicity. *Journal of statistical physics*, 104:817–879, 2001.

[3] Claude E Shannon. A mathematical theory of communication. *The Bell system technical journal*, 27(3):379–423, 1948.

Figure 3: The Architecture of Chaos



Top: The bifurcation diagram (values of x).
Bottom: Structural complexity (C_μ).